

Guide du Dashboard ObRail Europe

Ce document explique clairement le fonctionnement du dashboard Streamlit du projet MSPR : pages, KPIs, filtres, sources backend et lecture des graphiques.

1) Objectif du dashboard

Le dashboard sert a :

- visualiser les donnees ferroviaires europeennes chargees dans PostgreSQL,
- comparer train vs avion sur la partie emissions CO2,
- suivre la qualite des donnees,
- fournir une vue decisionnelle exploitable en soutenance.

Le dashboard n'invente pas ses donnees : il consomme l'API backend FastAPI.

2) Architecture fonctionnelle

Flux global :

1. Base PostgreSQL (**obrail**)
2. API FastAPI (**backend/app/...**)
3. Dashboard Streamlit (**dashboard/app.py**)
4. Graphiques Plotly (**dashboard/components/charts.py**, **dashboard/components/map.py**)

Le dashboard appelle l'API via **dashboard/services/api_service.py**.

3) Endpoints backend utilises

Le dashboard lit ces endpoints :

- **GET /stats/trajets/count** -> total des trajets
- **GET /stats/gares/count** -> total des gares
- **GET /stats/lignes/count** -> total des lignes
- **GET /gares** -> donnees gares (nom, coordonnees, pays)
- **GET /stats/trajets/type** -> repartition JOUR / NUIT
- **GET /stats/emissions** -> valeurs moyennes CO2 train / avion
- **GET /stats/operateurs** -> volumes de trajets par operateur
- **GET /stats/trajets/map** -> segments cartographiques pour la carte

Ces appels sont centralises dans **api_service.py** et mis en cache avec **@st.cache_data**.

4) Sidebar et filtres

La sidebar contient :

- navigation entre les 4 pages,
- filtre de volume de gares affichees sur la carte :
 - bouton - Diminuer
 - bouton + Augmenter
 - jauge Nombre de gares affichees
- filtre multi-pays (optionnel) via `iso_pays`.

Logique :

- si aucun pays n'est selectionne -> toutes les gares,
- si un ou plusieurs pays sont selectionnes -> sous-ensemble filtre,
- puis echantillonnage selon la jauge.

5) Page "Apercu"

5.1 KPIs

- **Trajets** : volume total de trajets
- **Operateurs** : nombre d'operateurs actifs
- **Lignes** : nombre total de lignes
- **Pays couverts** : compteur de couverture geographique

5.2 Graphiques

- **Pie Jour vs Nuit** : repartition des trajets JOUR / NUIT
- **CO2 Trains vs Avions** : comparaison des emissions
- **Carte reseau** : gares + couche cartographique
- **Qualite des donnees (mini card)** : manquants/doublons (vue rapide)
- **Volume par operateur** : classement des operateurs par volume

6) Page "Reseau Ferroviaire"

But : analyser la structure geographique des donnees.

Contenu principal :

- KPIs reseau (gares, segments, pays),
- carte complete du reseau,
- histogramme de repartition des gares par pays (`iso_pays`).

Cette page est orientee "couverture territoriale".

7) Page "Impact Environnemental"

But : rendre la comparaison carbone train vs avion lisible.

Contenu principal :

- KPIs CO2 (train, avion, ratio, CO2 economise),
- graphique CO2 principal,
- economies estimees par operateur,
- bloc jour/nuit + impact associe.

Lecture metier :

- plus le ratio avion/train est eleve, plus le train est avantageux,
- l'indicateur "CO2 saved" materialise le gain environnemental.

8) Page "Qualite des Donnees"

But : verifier la fiabilite du jeu de donnees.

KPIs :

- completude globale,
- valeurs manquantes,
- doublons detectes,
- volume total d'enregistrements.

Visualisations :

- manquants par colonne,
- completude par colonne,
- couverture GPS des gares avec/sans coordonnees.

Note : les colonnes **latitude** et **longitude** ont ete sorties des barres de completude/manquants quand demande pour eviter du bruit analytique.

9) Fichiers dashboard importants

- **dashboard/app.py** : orchestration des pages, sidebar, filtres, mise en page
- **dashboard/components/charts.py** : pie, CO2, operateurs
- **dashboard/components/map.py** : rendu carte
- **dashboard/services/api_service.py** : client HTTP vers FastAPI
- **dashboard/config/api_config.py** : URL de l'API backend

10) Lancement

Backend

```
cd /home/johan/MSPR/backend
source ../venv/bin/activate
uvicorn app.main:app --reload --port 8000
```

Dashboard

```
cd /home/johan/MSPR/dashboard  
source ../venv/bin/activate  
streamlit run app.py
```

11) Checklist de verification rapide

Avant demo/soutenance :

- `http://localhost:8000/health` repond `{"status": "ok"}`
- dashboard charge sans erreur
- les KPIs se remplissent (pas de valeurs nulles partout)
- la carte repond au filtre (jauge + pays)
- les pages `Apercu`, `Reseau`, `Impact`, `Qualite` affichent du contenu

12) Message soutenance (resume court)

Le dashboard ObRail Europe est une couche de visualisation adosse a une API FastAPI elle-meme connectee a PostgreSQL.

Il permet de suivre les volumes de donnees ferroviaires, l'impact CO2 train vs avion, la couverture reseau et la qualite de donnees, avec des filtres interactifs exploitables en contexte decisionnel.